iPRA: a 529-byte complete checker

Neutral premises, local scopes, and the remaining axiomatization problem

Research prepared for Vladimir Reshetnikov with ChatGPT

10 November 2026

Abstract

The requested significant new axiomatic breakthrough has not been established. This report gives the strongest verified result of the investigation: a complete checker of the inherited, full first-order intuitionistic PRA presentation, measuring 529 normalized Python bytes, against the 542-byte minimum in the current inspected corpus. An ordinary explicit-list implementation measures 537 bytes. An auxiliary implementation lowers the separate AST minimum from 291 to 290 nodes, with its inherited malformed-input qualification. No single program improves all three byte, token, and AST coordinates.

The reduction composes mechanisms already present in different drafts: verified neutral major premises, lexical fresh-name scopes, and shared construction of the continuation hypothesis. It changes the implementation, not the arithmetic axioms. The report states every native rule, proves the composition by simulation, records the complete source comparison and actual process tests, and reproduces the attributed mathematical bridge to iPRA. Both numerical interpretation composites are the identity on the whole domain. The written metatheorems are not proof-assistant certifications.

Claim boundary. There is a modest, reproducible improvement below the observed byte frontier. There is no established new simplest axiomatization, absolute size lower bound, or substantial removal of a logical or arithmetic primitive. The distinction is part of the result, not an omitted proof obligation.

Contents

1 The comparison and the precise result **3**

2 The complete native axiomatization being implemented **4**

3 The complete primary program **6**

4 Why the implementation composition is valid **7**

5 Bi-interpretability, rather than just numerical agreement **9**

6 Validation and reproducibility **10**

7 Unresolved reductions and further research questions **11**

Attribution and use of the inherited appendices **12**

A Complementary introductions with one local assumption **12**

B Internal scalar predecessor bootstrap **17**

C From literal-equation induction to full intuitionistic PRA **21**

D Pairing, primitive recursion, and internal numerical composites **27**

E Arithmetic strength and both ordinary interpretation composites **28**

F A uniform arity encoding and its checking theorem **30**

G Using the initializer to select computation **36**

H Why the absent major can be a proved empty array **39**

1 The comparison and the precise result

The repository snapshot is

`VladimirReshetnikov/ProveIt b9c4d74a0018bdafc5bc06e755a0145f4ed2b8d6`.

Its `docs/drafts` directory contains 20 direct ZIP archives. Every downloaded archive was checked against its Git blob SHA-1 and byte length. Recursive inspection finds 35 distinct archive payloads and 484 distinct Python sources, with no parse failures. All 41 sources at or below one of the observed frontier coordinates were classified: three are complete theorem checkers, and 38 are build, measurement, packaging, testing, or other supporting utilities. Complete checkers in reference and ancillary directories are included. The census, classifications, hashes, and exact source bytes are provided for offline reproduction.

Complete implementation	Bytes	Tokens	AST	Gzip 9
Archived byte leader	542	263	309	343
Archived token leader	578	261	295	346
Archived AST leader	642	263	291	353
`code/checker.py`	**529**	266	310	333
`code/checker_explicit.py`	537	266	308	337
`code/checker_ast.py`	610	263	**290**	345

Bytes are normalized UTF-8 source bytes. These six programs have identical raw and normalized lengths. The compact primary program saves 13 bytes, approximately 2.40%, relative to the archived byte leader. The explicit-list program is five bytes below that same compact archived leader; against the archived explicit-list implementation of 550 bytes it also saves 13 bytes. Its list test is preferable for ordinary source inspection.

The union of the old corpus and the delivered programs has coordinate minima

$$(\text{bytes}, \text{tokens}, \text{AST}) = (529, 261, 290).$$

The token minimum remains the old program's 261. This tuple is not the size of one program. In particular, the 529-byte program does not improve the token or AST minima. A separately executable readable expansion has 2,365 raw bytes and 1,841 normalized bytes; it is not an uncharged dependency of any compact program.

These are achieved minima in a finite corpus, hence upper bounds on an unknown optimum. They are not mathematical lower bounds on all possible axiomatizations or implementations. This is the operational reading of the requested comparison; absolute minimality is not asserted.

1.1 What is charged

The measurement removes genuine module, class, and function docstrings, uses `ast.unparse` and one final newline, and counts the entire module. Significant tokens exclude comments, indentation, line breaks, and end markers; AST nodes are the nodes visited by `ast.walk`. Imports, JSON input, scope validation, both arithmetic equations, every logical rule, and the verdict driver are included. The common Python runtime and its standard JSON parser are excluded under the unchanged corpus convention.

The recorded runtime is CPython 3.12.14 with zlib 1.3.2. Gzip is informational only, using `gzip.com press(source,compresslevel=9,mtime=0)`, including the gzip wrapper. Both raw and normalized

gzip measurements and the actual compressed sources are supplied. Different Python or zlib versions can change these implementation-specific measurements.

1.2 New composition, inherited mathematics

The neutral-major protocol comes from the monus and neutral-major draft; local scopes appear in the local-scopes and scalar-selfseed work; shared Boolean-chain hypothesis construction appears in the uniform-zero control. The appendix arithmetic, native logic, arity representation, zero repair, and whole-domain interpretation proof are inherited and attributed. The inspected corpus contains no byte-identical copy of the delivered primary program, but combining existing methods is not a new arithmetic foundation.

The recent self-seeded and monus arithmetic theories have different axioms. Their small accompanying programs implement initialized iteration. Their reported checker sizes therefore cannot be assigned to those alternative arithmetics without implementing and charging the missing rules.

2 The complete native axiomatization being implemented

Here iPRA means full first-order intuitionistic primitive recursive arithmetic: all fixed finite primitive-recursive descriptions, their defining equations, intuitionistic equality logic, quantifier-free induction, and the usual constructor axioms $Sx \neq 0$ and $Sx = Sy \to x = y$. The latter injectivity axiom also follows from the included predecessor equations. The nonzero-successor axiom is essential; positive equations alone admit a one-element structure. Merely matching its equations, equiconsistency strength, or standard-model numerical functions would not meet the requested bi-interpretability condition.

2.1 Numerical syntax and all nonlogical schemas

Numerical terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h).$$

Only the update body h binds v. Other variables are passive numerical parameters. Terms are finite staged descriptions, not codes supplied to an object-language universal evaluator. The two nonlogical computation schemas, available in every retained context, are exactly

$$\frac{}{\Gamma \vdash I(0, a; v.h) = a}\ I0, \qquad \frac{}{\Gamma \vdash I(Sn, a; v.h) = h[I(n, a; v.h)/v]}\ IS.$$

There are no separate primitive predecessor, pairing, equality-test, successor-injectivity, coverage, or separation axioms in the native calculus.

2.2 Formula syntax, contexts, and all inference rules

The essential object formulas are equations and one paired binder:

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A.$$

The implementation also uses the closure notation $\mathsf{Q}_y B(y)$ for the reusable conditional $B(0) \to \forall y B(y)$. It is a definable administrative formula, equivalent to $\mathsf{D}_{u,y} B(y)$ with unused fresh u; no separate primitive closure-introduction axiom is assumed. Contexts are finite reusable collections of formulas. In the following inventory Q abbreviations may be nested, with their definitional expansion understood.

$$\frac{}{\Gamma \vdash A}\ \text{Hyp} \quad (A \in \Gamma),$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \text{Tr},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}\ \text{QE},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \qquad \Gamma, \mathsf{Q}_y B(a,y) \vdash F}{\Gamma \vdash F}\ \text{DE},$$

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}\ \text{DI}_0,$$

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x,y)}\ \text{DI}_1 \quad (E \text{ a literal equation}).$$

In DE, a is fresh for Γ, F, and every fixed parameter of B. In either DI, z is fresh for Γ, the witness t, and every fixed parameter of the matrix. Transport uses any fixed capture-free formula template $F(w)$. All displayed local assumptions are discharged. A literal equation means an equation at the matrix root, not an equation beneath a binder.

Exchange, weakening, contraction, renaming, numerical substitution, and cut are admissible by induction on derivations; they are not extra instructions hidden in the checker. The public implementation initially retains the truth $0 = 0$, encoded by the empty array. This is a derived theorem and is discharged by cut. Appendix A gives a primitive clause-based presentation, proves this formula-context formulation equivalent, and derives the entire ordinary logical interface. Explicitly, eliminating $\mathsf{D}_{u,y} B(y)$ with unused u gives the clause $\langle B(0); y.B(y)\rangle$, from which QE follows by Use. Conversely, that clause proves $\mathsf{D}_{u,y} B(y)$ by case DI at an unused witness: under $B(0)$, use the clause at Sz. These conversions justify the formula-context DE resource and nested closure abbreviations without adding a logical axiom.

2.3 Why the restriction gives iPRA

The case rule supplies numerical zero/successor cases for arbitrary formulas. The progress rule, DE, and QE give contextual equation induction:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}.$$

Choose an unused existential coordinate, introduce its paired equation by progress, eliminate to $\mathsf{Q}_y E(y)$, and use the supplied base. This does not provide induction for arbitrary quantified formulas. Permitting compound progress would change the intended theory.

Equality reflexivity is derived from computation and transport: use $r = I(0, t; v.0)$, the computation $r = t$, and the transport template $w = t$ to infer $t = t$. Symmetry, transitivity, and congruence follow with their usual explicit equality templates.

The selector $C(n, a, b) = I(n, a; v.b)$ satisfies $C(0, a, b) = a$ and $C(Sn, a, b) = b$. It permits finite formula gates and selectors. These reconstruct intuitionistic implication, conjunction, disjunction, both quantifiers, and falsity from the paired binder. The arithmetic bootstrap then obtains predecessor, a characteristic of actual equality, quantifier-free induction, pairing, and arbitrary primitive recursion. These are established internal derivations, not additional primitives charged outside the program.

3 The complete primary program

The following is the entire 529-byte source. No theorem table, producer, normalizer, evaluator, or external mathematical library is imported.

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=0j, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d
    else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    c(m, g, k)
    a = m in ([s, u], not s and [t] * 3) and i(t, u) == a and i(t, s) or a
    h = m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

For readability, `code/checker_explicit.py` replaces only the test `a*0==[]` by `isinstance(a,list)`. For ordinary JSON values the two tests have the same successful container cases: lists pass; numbers, Booleans, and strings do not; null and objects cannot become accepted numerical scalars through the alternative test. The independently executable `code/readable_checker.py` spells out all operations and context updates.

The input is a JSON array. Its last item is the closed goal; preceding four-field packets $[m, s, u, t]$ are consumed backwards. An unused prefix is inert. Success means both exit status zero and exactly `ACCEPT` followed by a newline. Exceptions reject. Merely finding the word ACCEPT in diagnostic output is not a successful verdict.

3.1 Sorted representation

Equations are $[s, t]$, paired formulas are $[B]$, and canonical closures are $[B, B, B]$. Numerical terms use scalar zero, unary successor, and ternary iteration. At numerical positions an empty or binary array and arrays of arity at least four denote zero. At formula positions an empty array and arrays of arity at least four denote $0 = 0$. The first two fields of a general ternary formula are ignored annotations; the third is its closure body.

The opener uses the inherited common depth increment $5 - \mathrm{len}(a)$. Numerical root depth is three. Two scalar coordinates represent each protected bound slot, while negative coordinates above the current floor are rigid parameters. Successor has two dummy slots; iteration has one state slot in its update and one dummy slot in its other arguments. The full decoder and the semantic opening lemma are reproduced in Appendix F. The stipulated zero/truth meanings of surplus arrays are essential to soundness.

3.2 Packet meanings

Every packet first proves its major m. The canonical absent major is []; it closes immediately from the initial truth assumption, consumes no packet, and allocates no parameter.

If m is a nonempty array, matching an equality or a closure can replace the target by its transport or specialization premise. In all cases $m * 3$ is exported. A unary paired formula thereby supplies the intended closure at a fresh witness. Every other nonempty array exports a surplus-arity truth.

If $m = []$ and u is truthy, the packet supplies a computation. The field s is its predecessor, u its initializer, and t its update. A false-valued s selects zero computation; otherwise the counter is constructed as $[s]$. If $m = []$ and u is false-valued, the packet introduces a paired formula using witness t. Canonical $u = 0$ selects equation progress, and $u = []$ selects cases. An odd-arity compound matrix cannot obtain a progress assumption.

The syntactically truthy initializer restriction does not delete arithmetic instances. The established zero repair uses

$$C_0 = I(S(S0), S0; v.0), \qquad C_0 = 0.$$

Its computation has nonempty predecessor and initializer. Replacing a missing literal-zero initializer or predecessor by C_0 and transporting $C_0 = 0$ recovers the original computation. The full finite normalization, including its proof-node accounting, is reproduced in Appendix G.

4 Why the implementation composition is valid

The argument is a composition of verified transformations of the inherited checker. It is not a fresh proof of the predecessor bootstrap or a claim that finite testing establishes all arithmetic metatheorems.

4.1 Neutral premises are a semantic guard

The initial context contains [], denoting the derived truth $0 = 0$. All subsequent context entries remain lists. No numeric goal has a finite successful proof from such a context. To see this, take a smallest purported successful numeric-goal trace. Its initial membership test cannot close. A successful numeric major would be a smaller counterexample. Transport cannot change a numeric root to a list-valued formula, because root scalar opening is independent of its substitution image. A continuation would again be a smaller counterexample. Introduction cannot unpack a numeric goal and then obtain its length. These alternatives exhaust a successful trace.

Consequently a numeric-zero major cannot reach the unguarded expression `min([s,u],notsand[t]*3)`. This matters because its second candidate is Boolean false when s is truthy, and Python equates that value with numerical zero. The actual neutral major is an empty list, which does not have that equality. Appendix H supplies the full inherited raw-input argument, including numeric aliases and the mode comparison with u.

4.2 Local names replace a global allocation counter

Each activation owns its floor k. Proving its major passes the current floor to the recursive activation, which may allocate its own names but cannot change its caller's floor. The major and retained context were already checked under the caller's floor, so none of the major proof's private names occurs freely in its exposed conclusion or hypotheses. Reusing their labels after return is alpha-renaming of disjoint proof scopes.

The continuation decreases the caller's floor once. Both the successor goal and new assumption are opened after that decrease; the opener closes over the updated local variable. All context extensions create new lists. A subproof therefore cannot export private assumptions by mutating a shared context.

For integer coordinates there is one designated new eigenparameter, k. With accepted fractional coordinates there can be finitely many additional names in the newly exposed interval. They are all fresh for the retained target and context, and all must be included in the corresponding discharge. They are not silently identified with the designated integer name.

4.3 The hypothesis branch becomes one shared expression

The old three-way conditional for the supplied hypothesis can be written

$$m * 3 \quad \texttt{or} \quad (u \texttt{ and computation}) \quad \texttt{or} \quad \texttt{introduction}.$$

On a successful exact-list trace a substantive major is a nonempty list, and its triple is nonempty. A selected computation constructs a binary list, also nonempty. Hence the Boolean chain selects exactly the same first applicable branch as the old conditional. Introduction is reached exactly when both the major and initializer-or-mode field are false-valued.

This last condition also protects induction: no false-valued JSON value equals one, so the parity comparison cannot grant progress to a meaningful odd-arity compound matrix. The comparison must use u, not the unrestricted source field s.

4.4 Computation assumptions are discharged at zero

The shared continuation may retain an equation involving newly allocated names. Its soundness is not the assertion that this equation holds at every value of those names. The relevant admissible rule is

$$\Gamma, H(\vec{a}) \vdash F, \quad \vdash H(\vec{0}), \quad \vec{a} \notin \mathrm{FV}(\Gamma, F) \quad \Longrightarrow \quad \Gamma \vdash F.$$

Substitute zero for the finite family $\vec{a}$ in the first derivation and cut in the proof of $H(\vec{0})$. The old-floor scope check supplies the required freshness. The generated equation's zero instance is an $I0$ or IS instance. If the update uses its state, the inserted recursive source is rechecked at numerical depth three; if it does not, the update is constant in that state. The inherited arity proof treats ignored annotations separately. This is why the composition preserves computation soundness even for accepted aliases.

4.5 The missing-image sentinel

The fixed Python value `0j` is not a value supplied by the JSON reader. If an unresolved numerical hole attempts to validate it as an inserted image, the ordered comparison fails. This has the same successful evaluations as the former `None` sentinel. At formula padding positions no numerical image is needed. This representation choice is charged in the measured source; it is not an added arithmetic operation or a numeral of the object theory.

Theorem 4.1 (Composed checker)**.** *Every finite successful execution of the primary or explicit-list checker has a decoded conclusion derivable in the inherited native calculus. Conversely, every finite native derivation of a canonical closed formula has a finite accepted certificate under adequate execution resources. The public initial truth assumption is conservative. The two programs therefore present the same canonical closed theory as the inherited complete checker.*

Proof. For soundness induct on the finite successful call tree. Hypothesis closure uses the semantic opening lemma. A checked major permits only equality transport, closure specialization, paired

elimination, or adjoining truth. Introduction is exactly case DI or literal-equation progress DI, with the freshness established by the local floor. Computation uses the zero-instance discharge just proved. The neutral-major invariant validates the unguarded matcher; the shared expression selects the same rule as the old dispatcher. No child can mutate its parent's context or allocation state.

For completeness apply the inherited canonical proof compilation and certified-zero normalization, then rename every subordinate major proof's eigenvariables locally. Encode absent majors by [], use the matching case or progress mode, and consume the resulting finite packet tree in reverse stack order. A neutral major closes without consuming a packet. Every nonempty-major and computation export has the nonempty shape used by the shared expression. Thus every compiled proof follows the simulated trace. Finally replace uses of the initial $0 = 0$ by its finite native derivation. □

The primary opener terminates structurally on finite JSON syntax in the ideal adequate-resource reading: a substitution image is checked with the invalid default image, so insertion cannot start an unrestricted chain of further substitutions. Every nonclosing checker activation consumes a packet from a finite stack. Actual interpreters still have memory and stack limits.

4.6 The separate AST profile

The 610-byte profile replaces the tail-recursive continuation by a loop, using an immutable tuple context. Its recursively checked major still receives the old tuple. The augmented assignment creates a new tuple rather than mutating a context visible to another activation. This is an exact tail-call simulation and saves one AST node against the archived AST leader.

That profile also uses the inherited generic iterable opener. Empty strings and objects have documented truth aliases. Nonempty strings can recurse without termination in an abstract unlimited-stack interpreter; actual CPython rejects them through its recursion limit and subsequent type failure. The raw soundness statement concerns finite successful traces. This qualification does not apply to the primary or explicit-list opener. The smaller AST count is not a claim of an equally small structurally terminating parser on all finite malformed JSON.

5 Bi-interpretability, rather than just numerical agreement

Let T_I use ordinary intuitionistic first-order equality logic, the same staged iterator equations, contextual literal-equation induction, numerical coverage, and $(0 = S0) \to \bot$. The native realization is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

with $\mathsf{Q}_y B$ read as the displayed conditional universal. Case introduction is valid by coverage; progress introduction is valid by equation induction under the conditional base. The remaining native rules have their direct ordinary meanings. In the reverse direction, the formula gates and selectors derive every ordinary logical rule, coverage, equation induction, and falsity elimination. Their formula composites are proved in Appendix A.

Theorem 5.1 (Inherited full-domain equivalence)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable and equiconsistent. Both numerical interpretation composites have the internally definable identity isomorphism $x = y$ on the entire domain. The delivered checker presents this same theory on canonical encodings.*

Proof. An initialized scalar iterator is primitive recursive, so iPRA interprets every fixed iterator description and proves its defining equations and the other axioms of T_I.

For the other direction, the scalar predecessor proof in Appendix B derives $P(0) = 0$ and $P(Sn) = n$ using only the available equations, cases, and literal-equation induction. Appendix C constructs a numerical characteristic reflecting actual equality for arbitrary domain elements and converts each fixed quantifier-free formula to a Boolean equation. It therefore recovers quantifier-free induction. Appendix D constructs pairing and compiles each finite primitive-recursive description into the scalar language.

Internal equation induction proves uniqueness of every fixed iterator and primitive recursion from its defining equations. External induction on finite descriptions shows that double translation is provably equal to the original function on the whole domain, including substitution coherence for staged update descriptions. No restriction to standard numerals is made. The native paired formula is internally equivalent to its interpreted existential, implication, and universal formula; structural induction supplies both formula composites. The explicit ordinary falsity bridge preserves contradiction. Effective translations of finite proofs give equiconsistency. The new program inherits these facts by the preceding consequence-relation simulation, not through a new arithmetic bootstrap. □

The appendices reproduce the complete attributed arguments because a claim that all standard primitive-recursive functions are representable would not by itself prove the required internal round trips.

6 Validation and reproducibility

The final offline suite exercises the actual public programs under ordinary Python and `-O`. It covers computation, zero repair, transport, closure specialization, paired introduction and elimination, contextual equation induction, invalid scopes, compound-progress attempts, malformed values, and the documented generic aliases. The named certificates are inherited logical examples with the appropriate neutral-major and lexical renumbering; they are not new arithmetic theorems.

Two deliberate unsound mutations isolate concrete proof obligations. Changing the initial context from `[[]]` to `[0]` admits a two-packet proof of $1 = 0$; the correct checkers reject it. Changing the loop's persistent tuple context to a shared mutable list admits $0 = S0$; the correct loop rejects the identical packet. These controls make the invariants testable rather than merely checking that arbitrary malformed data raises an exception.

The shipped suite passes 408 complete-process verdict expectations (156 expected accepts and 252 expected rejects), with zero mismatches, plus the four recorded mutation-control outcomes. It also remeasures all 484 frozen historical Python sources without executing historical source code. The development-stage family and independent review receipts contain overlapping checks; their counts are not added to the shipped-suite count.

The tests are finite implementation evidence. They do not formalize the universal soundness theorem, arithmetic bootstrap, nonstandard-model claims, or bi-interpretability in Lean or Rocq. Independent assistant reviews are included as reviews, not external human peer review or proof-assistant evidence.

From a fresh extraction:

```
python3 verify.py
python3 measure.py
python3 build.py
```

The first two commands need only Python's standard library and no network. The last needs PDFLaTeX and the standard packages listed in the source. The expanded `ipra_neutral_scope.tex`

is independently compilable and contains all mathematical appendices and the checker listing. The payload manifest records the distributed files; rebuilding can change PDF metadata, so use a working copy after verifying the original archive.

7 Unresolved reductions and further research questions

The substantial search did not produce a new complete axiomatization below the current one. Deleting atomic case introduction and retaining only atomic progress is already refuted in the drafts; the all-starts repair is also already developed there. These rediscoveries are not claimed as results of this report. Recasting a checker as formula synthesis introduces return-rule and eigenvariable-escape obligations; no smaller complete version was obtained.

The scalar self-seeded candidate

$$F(0; v.h) = S(h(0)), \qquad F(Sn; v.h) = h(F(n; v.h))$$

remains structurally interesting. Its existing treatment explicitly adds an equality characteristic and successor injectivity. The initial zero test does not by itself justify arbitrary numerical selection or a source-language compiler for graph-defined functions. Omitting those obligations would make the apparent simplicity comparison invalid.

The next substantive questions are therefore:

1. Can a logical or arithmetic primitive actually be removed while preserving the full native formula composites, rather than merely its standard-model equations or consistency strength?
2. Can the self-seeded basis derive its additional equality and constructor principles, or can a complete implementation charge them and still beat the initialized-iterator checker?
3. Can the semantic opener, fractional-name discharge, and computation source check be formalized in Lean or Rocq without replacing the actual raw-input program by an easier canonical-only model?
4. Can a fixed implementation language, input contract, and explicit allowed class of presentations support a meaningful lower-bound theorem? A corpus census alone cannot provide one.

The present deliverable is a verified implementation reduction and an exact account of what remains open. It does not resolve the requested significant axiomatic breakthrough.

Attribution and use of the inherited appendices

The following eight source fragments are reproduced from the pinned ProveIt draft corpus with their original mathematical proofs and labels. Their historical phrases such as “new” describe their original reports, not the current contribution. Source identities are recorded in `evidence/provenance.json`; the upstream MIT No Attribution license is included in `reference/UPSTREAM_LICENSE.txt`. Mathematical attribution is retained independently of that license.

The first five appendices supply the native basis and full arithmetic bridge. The next supplies the inherited sorted decoder and its older packet protocol. The final two supply the subsequent certified-zero and neutral-major changes. Section 4 proves how the current local-scope and shared expression composition transfers their results. Historical packet-field conventions in an appendix must not be mistaken for the current interface in Section 3.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use}\quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A, B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [3] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m), u, Su), \qquad W(t,m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su,m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0,m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t,m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t,m)) \\ &= SB_m(W(t,m)) = SW(St,m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n,n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

C From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [4]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix B, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

C.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

C.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

C.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

C.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

C.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

C.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at

zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

D Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [5]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

D.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k + 1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

D.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

D.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\perp$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

E Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u,\vec{p})$, introduce $i_h(n,a,\vec{p})$ with the two equations

$$i_h(0,a,\vec{p}) = a, \qquad i_h(Sn,a,\vec{p}) = h(i_h(n,a,\vec{p}),\vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

E.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem E.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

F A uniform arity encoding and its checking theorem

The arithmetic and object logic remain the scalar-iteration Eq/D calculus specified above. The change is in its finite certificate representation. A paired formula is unary, an equation is binary, and an internal closure is ternary:

$$\mathsf{D}_{x,y}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s,t], \qquad \mathsf{Q}_y B \longleftrightarrow [B,B,B]. \qquad \text{(Arity encoding)}$$

The three copies in a closure are representation redundancy: only the third has mathematical content. Their benefit is that tripling a unary major constructs exactly its closure. Tripling any other nonempty formula array produces an array of arity at least six, which will denote truth. This makes the other major cases neutral without a separate test.

The previous generated-assumption and constructed-computation interfaces are inherited from the corpus [2, 6]. The new encoding makes a single arithmetic expression, $5 - \text{arity}$, perform the necessary depth adjustments. Its soundness depends on an explicit interpretation of surplus syntax. It would not be sound to drop arity checks while continuing to treat every surplus array as an unconstrained new connective.

F.1 The complete sorted interpretation of arrays

Write $\top$ for the fixed equation $0 = 0$. A protected numerical slot has width two. The numerical root has depth three: the coordinates $0, 1, 2$ all denote zero, and under p protected slots the depth is $3 + 2p$. Coordinates $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot, for $0 \le j < p$. Formula depth is $2p$ under the same p slots. Negative coordinates admitted by the current global floor name rigid numerical parameters.

The decoder is sorted by the position in which a tree is used. Let ρ be its ordered environment of protected numerical slots. The notation (a, ρ) means prepend a slot of value a. A superscript $\sharp$ denotes a child decoded in the indicated environment.

Numerical arity	Meaning	Relevant child environment
0	0	No child
1: $[b]$	$Sb^\sharp$	$(0, 0, \rho)$
2	0	Annotations ignored
3: $[n, a, h]$	$I(n^\sharp, a^\sharp; v.h^\sharp)$	$(0, \rho)$ for n, a; (v, ρ) for h
≥ 4	0	All children ignored

Formula arity	Meaning	Relevant child environment
0	$\top$	No child
1: $[B]$	$\mathsf{D}_{x,y}B^\sharp$	(y, x, ρ)
2: $[s, t]$	$s^\sharp = t^\sharp$	ρ numerically
3: $[P, U, B]$	$\mathsf{Q}_y B^\sharp$	(y, ρ) for B
≥ 4	$\top$	All children ignored

The first two entries of a ternary formula are annotations. A canonical closure copies its body into both of them, as in (Arity encoding). Successfully admitted scalar formula padding also denotes $\top$. An internal closure has the ordinary meaning $B(0) \to \forall y B(y)$; it is not an equation and is not eligible for compound induction.

Canonical encoded object expressions use arrays and integer coordinates, respect these slots, and contain no high-arity node. The checker itself still generates high-arity truth entries in administrative contexts, even during a canonical proof. In particular, constructing a successor beneath positive bound coordinates requires passing through two dummy zero slots. Constructing an iterator requires one new slot: zero in its counter and seed, and its genuine state variable in the update. Rigid negative parameter names are unaffected by these shifts.

The additional raw shapes are certificate aliases with the displayed meanings. They are not independent predicate or function symbols added to the object theory. Our exact completeness statement concerns the canonical native language. We do not require every possible raw spelling of a semantic alias to be interchangeable under the same raw certificate.

F.2 One depth calculation replaces the lookup table

For every array the opener recursively adds $5 -$ arity to its depth. The meaningful cases are

arity	1	2	3
depth increment	4	3	2.

Unary D adds two numerical binders; unary successor adds two dummy zero slots. Binary equality changes formula depth $2p$ to numerical depth $3 + 2p$. Ternary Q adds one numerical binder; ternary iteration adds one state slot, or one dummy slot in its counter and initializer. The same calculation therefore covers all relevant constructors.

Binary numerical annotations denote zero, so their parity-changing traversal has no mathematical effect. Empty arrays have no children. At arity at least four, the entire node already denotes a constant.

Even a zero or negative depth increment below such a node cannot expose one of its ignored children as a meaningful variable. Traversal may still reject an alias; it cannot change its stipulated interpretation.

This is a semantic assertion, not raw idempotence of the opener. For example, at floor -1, opening a six-element array of zeros with image -1 can yield six copies of -1, while a later no-image opening of that result fails at a negative child depth. Both formula arrays denote $\top$. The proof below only concerns successful calls at meaningful sorted positions. Canonical input object expressions contain no surplus arrays; the administrative exports that generate them are justified separately as truth entries.

F.3 Opening and scope

Write $i(a, t, d)$ for the executable opener at floor k. It preserves a scalar when $k \leq a < d$. At a numerical position, every other relevant scalar is the distinguished substitution hole; the supplied image is checked at numerical root depth three before insertion. At a formula position scalar padding is truth and carries no numerical hole. Missing images at used numerical holes reject.

Lemma F.1 (Semantic opening)**.** *For every successful call at a meaningful formula root of depth $2p$ or numerical root of depth $3 + 2p$, with $p \geq 0$, decoding its result is capture-free filling of the distinguished hole in the decoded input template. These depths protect p numerical slots; admitted rigid parameters are preserved. If an image is semantically unused, zero may be chosen as its mathematical representative. Recursive calls at negative depths inside ignored annotations need no independent sorted interpretation.*

Proof. Induct on the finite raw tree. At a relevant scalar, the interval comparison is exactly the protected-coordinate test. At a meaningful unary, binary, or ternary node, the preceding tables supply precisely the required additional slots. At numerical binary annotations, high-arity nodes, and ignored closure fields, the relevant mathematical meaning is constant or absent, so the conclusion is immediate regardless of traversal inside the annotation.

An inserted image is checked at its own numerical root. Its free parameters are rigid negative names, and it has no unbound positive coordinate there. Insertion beneath protected slots therefore captures none of its parameters; its internal binders retain their relative indices. This proves the inductive step for substitution as well as for mere traversal. □

A successful no-image check consequently preserves the interpretation of its checked closed expression. Raw structural equality of checked outputs implies equality of their decoded meanings. The checker does not infer equality merely from a semantic comparison of two different programs or numerical terms.

The global floor decreases by one at each non-hypothesis continuation. If its old value is K, the integer $K - 1$ is the designated fresh eigenparameter. With integer coordinates it is the only new name. If fractional coordinates are accepted, other names in $[K - 1, K)$ may also occur in the finite certificate. These are additional fresh parameters. They must not be silently equated to the designated integer eigenparameter.

Previously checked relevant goals and retained hypotheses mention none of these new names: their earlier no-image checks would reject a relevant numerical occurrence below the old floor. Coordinates inside ignored annotations do not contribute free variables to the decoded formula. This is the freshness invariant used in every rule below.

F.4 A computation hypothesis only needs a zero instance

The shared continuation can open a generated arithmetic equation at a fresh parameter. That equation need not be a theorem uniformly in the new parameter. The following admissible rule is sufficient.

Lemma F.2 (Supply with a known zero instance)**.** *Suppose $\vec{a}$ is fresh for Γ, F, the calculus proves $H(\vec{0})$, and $\Gamma, H(\vec{a}) \vdash F$. Then $\Gamma \vdash F$.*

Proof. Substitute zero for every member of $\vec{a}$ in the second derivation. Freshness leaves Γ, F unchanged. Cut in the derivation of $H(\vec{0})$. Both substitution and cut are admissible in the native calculus. This is a finite proof transformation and does not use a consistency assumption. □

All names newly admitted when the floor changes belong to $\vec{a}$; for noninteger aliases this includes the finite collection in $[K-1, K)$ that actually occurs. On decoding, their simultaneous zero substitution agrees with opening the generated expression at zero using the old floor. Previously admitted parameters are preserved. New rigid names and unprotected placeholders both become zero. The opener may keep different raw zero spellings, but their decoded values agree.

The constructed-computation packet supplies a triple $s = [n, b, r]$ and a mode u. In successor mode its generated equation has raw form

$$\big[\,[[n], b, r],\ i(r, s, 3)\,\big], \tag{Step supply}$$

while in zero mode it has form

$$\big[\,[u, b, r],\ b\,\big]. \tag{Zero supply}$$

Here successor mode means that u is truthy; the zero mode has falsy u. These expressions describe actual arrays, before their final opening. In (Step supply) the counter is constructed as a successor, rather than accepted on an unsupported assertion that it is one. This construction is inherited from the corresponding draft interface [6].

Lemma F.3 (Sound constructed computation)**.** *Every successful arithmetic supply has a decoded zero instance derivable from the two scalar-iteration computation schemas.*

Proof. First apply the simultaneous zero substitution described above. In zero mode every successful falsy numerical counter denotes zero, so (Zero supply) is an $I0$ instance. In the exact-list profile the relevant accepted aliases are an empty array, numeric zero, false, and numerically equal zero values. Null, strings, and dictionaries cannot enter a meaningful numerical position through an unchecked coercion.

In successor mode the left counter is an actual successor. If the update uses its state in a semantically relevant position, opening it invokes the no-image check of the recursive source s at numerical root three. This checks the entire source, including its update. Only coordinates three and four are aliases for its single state slot. A genuinely escaping coordinate in that update is revisited during the recursive-image check and rejects. Its counter and initializer are likewise closed with their prescribed dummy zero slot.

Inserting the extra dummy slots of the displayed successor does not alter such a checked predecessor: extra unbound slots have value zero, and actual free parameters are rigid. Lemma F.1 therefore identifies the right side of (Step supply) with the update applied to exactly the recursive predecessor value occurring on its left. This is the IS schema.

If the update has no semantically relevant state occurrence, its decoded value is constant in that variable. Its zero-normalized value is the same in the source update and on the right, independently of the unused recursive image; this is again an IS instance. Traversal inside an ignored annotation may

still inspect the image and reject. That removes a certificate and does not invalidate the argument for a successful one.

Finally, successful three-field source extraction cannot hide another JSON operator. A scalar is not iterable. A three-character string, or a three-key JSON dictionary, necessarily puts a nonempty string in a relevant source field, which the inspected opener rejects. The successful source is therefore the stated array triple. □

The recursive-image check in the state-dependent case is essential. Merely opening a final displayed equation at zero would not establish that its two state interpretations agree. Combining Lemma F.3 with Lemma F.2 justifies both actual image choices of the common continuation, including the branch that retains a fresh-parameter hypothesis.

F.5 Audit of the logical continuations

Hypothesis closure. The initial opener check and membership comparison select the same decoded context formula. Additional aliases can make a spelling less convenient, but cannot identify an equation with a false denotation solely by semantic evaluation.

Equality transport. The major must match $[s, u]$, and opening the proposed formula template at u must match the target. The continuation checks its opening at s. Lemma F.1 gives ordinary transport along the decoded equality. The argument applies to compound formulas, internal clauses, and truth aliases as well as equations.

Closure specialization. The closure pattern is the triplicate $[t, t, t]$. The guard forces the old instance to have numerical image zero; the new instance may have an arbitrary numerical image. This is exactly $B(0) \to \forall y B(y)$. The zero guard is indispensable. Without it, a closure with false zero instance and true successor instances would turn a proved successor instance into falsity.

Paired elimination. The export is always three concatenated copies of the major. A unary major $[B]$ therefore exports $[B, B, B]$. All three copies are opened identically; the third copy has precisely the witness instance required by paired elimination. The newly allocated witness is fresh for the retained context and conclusion.

An equation major instead exports arity six; a closure major exports arity nine; a larger array exports still larger arity. These are all truth aliases. Thus failed elimination endpoint claims have only the effect of adjoining a theorem. A numeric scalar major, if accepted as formula padding, exports scalar padding after multiplication and also denotes truth. Unsupported types reject.

A ternary formula with unequal annotations still denotes the closure of its third child. It fails the triplicate specialization pattern unless the three fields match. This loses a possible raw spelling, not a mathematical soundness condition. Canonical compilation uses triplicate closures throughout.

Paired introduction. The call `i(*a,t,2)` uses the function's argument count to enforce the relevant outer arity. For a unary goal $a = [B]$, it is exactly `i(B,t,2)`: the universal slot is protected while the existential witness is opened. For arity at least two, there are too many arguments and the call rejects. An empty goal can instead make the legal two-argument call `i(t,2)`. Its original target was already $\top$, so accepting it cannot add a false theorem.

For the unary goal, the subsequent successor opening gives the intended premise $B(t, Sz)$. The local hypothesis is $B(t, 0)$ in case mode and $B(t, z)$ in progress mode. The freshness invariant supplies the usual eigenvariable conditions.

Induction remains restricted. Among meaningful nonconstant matrix constructors, only binary equality has even arity. Unary paired formulas and ternary closures have odd arity and receive case introduction. Empty or surplus arrays that reach the progress branch denote the fixed equation $0 = 0$, whose induction is valid. Scalar padding that cannot provide a length rejects. The broader certificate alphabet therefore adds no induction on a quantified matrix.

Additional fresh parameters. An accepted subderivation can contain newly admitted fractional names, but the older context and target do not depend on them. Replace these auxiliary names by zero when closing the rule, retaining its designated integer eigenvariable where that rule needs one. The common opening of an already checked retained goal leaves its decoded meaning unchanged. This handles the actual global-floor convention without pretending that every admissible negative number is an integer.

F.6 Canonical completeness, including zero witnesses

Rename a finite native proof's eigenvariables to fresh negative integers. Encode its terms with the slot conventions of Section F.1; encode paired formulas as unary and closures as triplicate. Hypothesis leaves consume no instruction. Transport, closure specialization, and paired elimination have their displayed packets. Arithmetic computation leaves use (Zero supply) or (Step supply) and then close the target by hypothesis lookup.

One representational issue requires an actual proof transformation. The packet's truthy witness field selects introduction. A literal zero witness therefore cannot simply be inserted into that field. Define

$$J(t) = I(0, t; v.0).$$

The computation axiom proves $J(t) = t$, and $J(t)$ has a nonempty array representation even when its value is zero. To introduce with a falsy witness spelling t, introduce instead with $J(t)$: transport its retained assumption to the assumption using t, use the original premise derivation, and transport the resulting successor instance back to the instance using $J(t)$. Derived symmetry supplies the reverse transport. This is a finite transformation of the premise, and the paired conclusion itself is unchanged. If the existential coordinate is unused, any closed truthy witness works without a conversion.

The scalar coordinate 1 is also a semantic zero at numerical root three, but that observation alone is not a completeness proof. Closure specialization and case introduction insert literal raw zero, and distinct zero aliases need not be interchangeable under a fixed raw proof spelling. The computation-backed box gives the required derivation instead of assuming representational interchangeability.

The mode zero selects progress at an equation. A different falsy mode, such as the empty array, selects case. These are certificate control tags, not numerical assertions about the witness. All steps in the compilation are finite and effective.

Theorem F.4 (Conservativity and completeness of the arity checker)**.** *Every accepted certificate has its decoded conclusion provable in the ordinary realization of the native calculus. For every canonical native end sequent, the internal checking judgment and the native calculus have the same consequence relation, in the idealized unbounded integer-and-array implementation. In particular, the public empty-context driver recognizes exactly the canonical closed theorems. The translation in each direction is effective on finite derivations.*

Proof. For soundness, induct on the finite accepting call tree. Hypothesis closure uses Lemma F.1; the logical cases are the audited continuations above. Arithmetic supply uses Lemmas F.3 and F.2. Every discarded annotation is accounted for by its zero or truth interpretation, and every fresh parameter by the scope invariant. For completeness, compile a finite native derivation by the preceding construction, including the boxed-witness transformation. Its canonical input expressions require no surplus-arity aliases or fractional names; generated administrative surplus arrays have exactly the truth meaning already proved. This proves both implications without using a finite test suite as a substitute for induction on derivations. □

The unchanged native calculus is bi-interpretable with full first-order iPRA by the inherited arithmetic and logical bridge [2, 1]. In particular,

$$(\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x,0) \to \forall y B^\circ(x,y)).$$

The whole numerical domain and actual equality are retained. Its internally proved numerical round trips and its formula-composite derivations therefore transfer through Theorem F.4. We have changed the checked representation, not replaced bi-interpretability by standard-model agreement or supplied a new proof of the scalar predecessor theorem.

F.7 JSON and finite-runtime qualifications

The primary profile uses an ordinary list-type test. The smaller byte profile instead tests multiplication by zero against an empty list. Among standard JSON runtime values the latter recognizes exactly lists: numeric values and booleans produce numeric zero, strings produce an empty string, and null and dictionaries raise rather than becoming arrays. This argument assumes ordinary decoded JSON values, not arbitrary Python objects with overloaded methods.

Finite nonnegative real coordinates have the same interval interpretation: $[0,3)$ is zero, and $[3+2j, 5+2j)$ names slot j. Finite negative coordinates in the admitted range are additional rigid names. Equal Python numerical values have the same decoded role, including Boolean aliases and signed zero. Nonfinite values admitted by Python's permissive JSON reader cannot survive a relevant closed numerical check. They may be filled as template holes or ignored inside a constant annotation; neither makes them unchecked parameters.

The alternative opener based on generic iteration has a separate runtime qualification. Empty strings and empty dictionaries traverse as empty arrays; a dictionary whose sole key is empty traverses as a unary array with an empty child. Dictionary values are not visited. Nonempty strings can rely on CPython's recursion limit to reject, as also occurs in a current corpus comparison profile [7]. The idealized structural-termination claim for the exact-list profile must not be transferred to that generic profile unchanged.

All executable profiles have ordinary finite memory, stack, and input limits. These can reject a sufficiently large otherwise valid proof. The formal consequence relation is the unbounded finite-certificate calculus; the delivered program checks certificates within its runtime resources. The measurements charge the entire delivered checker, while the semantic proof and canonical producer are supplied separately.

G Using the initializer to select computation

The initialized scalar iterator has the usual equations

$$I(0,a;v.h) = a, \qquad I(Sn,a;v.h) = h[I(n,a;v.h)/v]. \tag{I0, IS}$$

The following restriction removes a computation opcode from the certificate interface without changing these operations or the native consequence relation. It concerns the *syntax* of an initializer, not whether that initializer denotes a positive natural number.

A fixed nonempty representation of zero. Let

$$C := I(S(S0), S0; v.0). \tag{Auxiliary zero}$$

One successor computation, at predecessor $S0$, proves

$$C = 0. \tag{Zero certificate}$$

The predecessor and initializer in this computation are both nonempty canonical terms. The update is the constant zero, so the right side of the computation is exactly zero. This construction does not use a previous computation with initializer zero, a zero-predecessor repair, an induction, or a presumed identity for a new numerical constructor.

Restricted computation system. In canonical numerical serialization, use scalar 0 for zero and nonempty arrays for successor and iteration. Consider the same native calculus with its direct computation leaves restricted as follows:

- I0 may be used directly only when its initializer is not literal zero.
- IS may be used directly only when both its initializer and its supplied predecessor are not literal zero.

The predecessor restriction is already present in the inherited predecessor-specified interface. The new restriction is on the initializer. A parameter or a nonempty term is permitted even when its value happens to be zero. In particular, C is permitted. Keep the ordinary native equality-transport rule and all other logical rules unchanged.

Admissibility theorem. Every unrestricted I0 and IS instance has a finite derivation in this restricted system. Consequently the restricted and unrestricted calculi prove exactly the same native sequents, with exactly the same numerical and formula languages.

Proof. Instances satisfying the stated syntactic conditions are direct. For a base computation with initializer zero, take the fresh-variable equation motive

$$E(w) := \big(I(0, w; v.h) = w\big).$$

The restricted base equation proves $E(C)$. Equality transport along the independently proved $C = 0$ gives $E(0)$, which is precisely the required unrestricted base instance.

For a successor computation with permitted predecessor n but zero initializer, take

$$E(w) := \left(I(Sn, w; v.h) = h[I(n, w; v.h)/v]\right).$$

The restricted successor equation proves $E(C)$, and the same transport gives $E(0)$. The displayed variable w is fresh for n, h, and all retained assumptions. Only the indicated initializer occurrences are substituted; unrelated zero occurrences or passive arguments in h remain fixed.

For a zero predecessor but a permitted initializer a, use instead

$$E(w) := (I(Sw, a; v.h) = h[I(w, a; v.h)/v]) .$$

Again $E(C)$ is a permitted successor computation and transport gives the desired instance $E(0)$.

If both predecessor and initializer are zero, a single shared motive repairs them simultaneously:

$$E(w) := (I(Sw, w; v.h) = h[I(w, w; v.h)/v]) .$$

Both supplied values in $E(C)$ are nonempty, and transport along $C = 0$ gives exactly $E(0)$. Bound state variables are renamed before these substitutions, so no capture occurs.

Each repaired computation leaf therefore uses one permitted computation for $C = 0$, one permitted computation for $E(C)$, and one transport inference. Replacing computation leaves in a native derivation proves the inclusion from the unrestricted calculus to the restricted one; the reverse inclusion is immediate. Contexts and final sequents are unchanged throughout. □

This is a three-node repair when computation leaves and transport are counted as one node each and the proof of $C = 0$ is copied. If a tree derivation has N nodes and r excluded computation leaves, the stated replacement has exactly $N + 2r$ nodes. If a DAG proof retains one shared node proving the closed lemma $C = 0$, the repaired part instead has $2r + 1$ nodes, giving $N + r + 1$ distinct nodes when $r > 0$. A certificate format with persistent proved assumptions can implement corresponding sharing by supplying this closed computation once before the affected continuation. These are counts for the displayed construction, not lower bounds on shortest certificates or claims about a particular producer's serialization.

Why the opcode can disappear. Write a four-field packet as $[m, s, u, t]$. The established elimination branch uses a truthy major m. In a computation packet, let s be the predecessor flag/value, u the initializer, and t the update. A truthy u selects the computation branch. The checker constructs the source counter as s when it is the zero flag and as Ss otherwise, and generates I0 or IS. A paired-introduction packet puts zero in the unused u field. Thus the dispatch can be written

```
if m:
    ... elimination ...
elif u:
    ... computation ...
else:
    ... paired introduction ...
```

The direct computation branch can no longer receive a literal-zero initializer. The admissibility theorem supplies exactly that missing case. No test that u denotes a nonzero number is performed or needed.

The fresh-name obligation on noncanonical packets. The dispatch change admits computation packets with any false-valued m, including the atomic-progress marker $m = 0$. In the common continuation, that marker can cause the generated hypothesis to be opened at the new rigid parameter b, rather than at literal zero. It would be incorrect to assert that the resulting $H(b)$ is uniformly true. Use the existing zero-instance invariant instead:

$$\vdash H(\vec{0}), \qquad \vec{b} \cap \mathrm{FV}(\Gamma, A) = \varnothing.$$

Here $\vec{b}$ includes the designated new integer name b and every additional negative fractional name in the newly admitted unit interval that occurs in the finite raw certificate. Such names must not be

silently identified with the designated integer. Given a finite derivation of $\Gamma, H(\vec{b}) \vdash A$, rename its later eigenparameters if necessary, substitute zero for every member of $\vec{b}$, and cut the resulting assumption $H(\vec{0})$ against its computation derivation. This gives $\Gamma \vdash A$. The old-floor validation of the retained goal and context is what establishes the displayed freshness condition. Consequently the new raw packet possibilities are sound under the same zero-completion and opening lemmas as the predecessor-specified kernel. They do not justify silently strengthening a known zero instance into a uniform computation theorem.

Interpretations and the scope of the improvement. The theorem preserves every native sequent through the identity map on object expressions. It therefore transfers the inherited whole-domain bi-interpretation and equiconsistency with iPRA without a new domain, new equality relation, or new primitive numerical operation. The contribution is a removable syntactic restriction and a data-driven certificate dispatch. It is an implementation simplification, not a new arithmetic foundation or a proof of absolute minimality.

Relative to the inspected pinned draft collection, the new step is using the initializer itself to discriminate computation packets and proving that the resulting missing computation instances are admissible. The broader technique of replacing an unavailable zero representation by a proved nonempty representative is already used in the supplied drafts, including their predecessor and witness repairs. The present argument adapts that attributed technique to eliminate the computation opcode. No priority claim outside the inspected corpus is made.

H Why the absent major can be a proved empty array

This section audits the new control-flow change on raw input, including noncanonical JSON. It uses the inherited sorted arity decoder and semantic-opening lemmas for the numerical and logical rules. It does not replace those lemmas with finite testing or claim proof-assistant certification.

H.1 The exact change

The initial context is the one-element list `[[]]`. Its sole formula, the empty array, denotes $0 = 0$ under the inherited decoder. At every non-hypothesis command the checker now executes

```
m, s, u, t = q()
c(m, g)
```

without testing whether `m` is nonempty. A canonical command with no substantive major puts `[]` in this field. Its premise closes immediately by hypothesis, without consuming a command or changing the fresh-name floor.

The final choice of the retained assumption uses

```
(m or len(a) % 2 == u) and k
```

instead of comparing the arity parity with `m`. In an introduction command, `u=0` selects equation progress and `u=[]` selects the zero-case premise. This reuse is legitimate because introduction is reached only when both `m` and `u` are false-valued. Computation still requires a truthy initializer `u`; its missing literal-zero cases have the inherited certified-zero repair.

The source is deliberately left with the old matcher

```
m in ([s, u], not s and [t] * 3)
```

and no added `m and` guard. As an isolated Python expression this would be unsafe: if `s` is truthy, its second alternative is `False`, which compares equal to numeric zero. The following whole-program invariant proves that this dangerous major cannot reach the matcher.

H.2 Two facts about the root opener

Call a Python value *numeric* here when it is an integer, a float, or a Boolean returned by the JSON loader. The finite Python execution also admits the loader's optional nonfinite float values. No custom numeric class is supplied by a certificate.

Write $O_k(r;t)$ for the opener on the numeric value r, with image t and its default formula-root depth zero. Direct evaluation of the program gives

$$O_k(r;t) = \begin{cases} r, & k \le r < 0, \\ 0, & \text{otherwise.} \end{cases} \qquad \text{(Root scalar)}$$

In particular this result is numeric and independent of the image t. The formula covers negative allocated names and negative fractional names without identifying different numerical values. Signed zero and Booleans have Python's ordinary numerical equality behavior. A NaN fails the interval test and returns zero; the relevant infinite values also return zero. None of these results is a list.

A second fact is that a successful root opening of a list returns a list. Under either strict list predicate, no other input type can return a numeric result except a numeric input. Under the generic iterable opener, a successful iterable traversal also returns a list; if traversal fails, the numerical comparison cannot succeed on a list, string, dictionary, or null. Thus in every profile:

> If a root opening returns a numeric result, its argument was numeric; if it succeeds on a list, its result is a list.

These statements concern finite successful evaluations. They assert neither arbitrary revalidation idempotence at ignored annotations nor unlimited-stack termination for the generic iterable profile.

H.3 No numeric goal can be proved from list-valued assumptions

Lemma. For any initial integer fresh-name floor and any finite remaining certificate, there is no finite successful invocation `c(a,g)` whose raw goal `a` is numeric and whose every context element is a list.

Proof. Suppose otherwise, and select such a successful invocation with the fewest calls to `c` in its completed recursive trace. Its first hypothesis check cannot close: the root opening of its numeric goal is numeric, and no numerical JSON value equals a Python list.

The invocation must therefore consume a command and successfully return from `c(m,g)`. If `m` were numeric, this would be a strictly shorter successful invocation of the forbidden kind. Thus the major is not numeric.

In either strict profile, a successfully checked nonnumeric major is a list. In the generic profile an empty string or an empty dictionary can also open to the empty list. A nonempty dictionary can occasionally have its iterable alias proved, but it cannot occur as a truthy major in the selected completed parent invocation: the subsequent expression `m * 3` raises a type error. A nonempty string has no finite successful opening. Consequently every truthy major that survives to the continuation exports a list, and every surviving false-valued major is an empty list or one of those harmless empty iterable aliases.

Consider the possible change of goal. If the matcher does not fire, the goal stays numeric. If it fires, its target test requires `i(t,u) == a`. The second root-opening fact therefore implies that `t` is numeric. By (Root scalar), `i(t,s)` and `i(t,u)` are the same root result. Both the Boolean-expression and the explicit-conditional implementations consequently retain a numeric goal. The Boolean implementation may retain the original raw zero alias when the transported result is false-valued; that makes no difference to this assertion.

If the major is truthy, the generated hypothesis is the list `m * 3`. If the major is false-valued and `u` is truthy, the computation expression constructs a two-element list. A successful opening of either hypothesis is a list. The final recursive call then has a numeric root-opened goal and an entirely list-valued context. It is a proper subtrace of the selected invocation, contradicting its minimality.

The only remaining case is introduction. Its expression `i(*a,t,2)` must unpack the numeric raw goal as an iterable, which raises a type error. That case cannot belong to a successful trace. All possibilities have been excluded. □

The minimization ranges over all starting floors and stacks. It is therefore unaffected by a recursively checked major decreasing the global floor before its parent resumes. The argument uses finite completed traces, not a presumption that recursive checking always terminates.

H.4 Every reached command has a list-valued context

Initially the only assumption is the list `[]`. Suppose a command is reached with list-valued assumptions and its major check returns. The lemma excludes a numeric major. A truthy surviving major exports a list; a computation constructs a list. In introduction the assignment is

```
h = a = i(*a, t, 2)
```

in mathematical order. If its result is numeric, the later `len(a)` raises a type error before the continuation context is constructed, since this branch has false-valued `m`. Such a scalar matrix does *not* insert a scalar-zero assumption. If the assignment yields a list, successful opening again yields a list.

Both recursive premises thus receive list-valued contexts whenever execution reaches them. The expression `g + [new]` constructs a new list; a premise cannot mutate its caller's context. Induction on the reached calls proves the invariant. The tuple-based explanatory variant has the corresponding immutability property.

It follows that no numeric zero, Boolean `False`, floating zero, truthy numeric scalar, or newly allocated numeric name can be a successfully verified major at a reached command. In particular a closed scalar-zero goal is rejected by this interface. The canonical empty-array truth is accepted instead; mathematical completeness is claimed for canonical encodings, not for every redundant truth alias.

H.5 The unguarded matcher is safe on every reached continuation

If `s` is false-valued, both matcher alternatives are nonempty lists, so no false-valued major can match. If `s` is truthy, the second alternative is `False`. A false-valued major reaching this point is an empty list, or in the generic profile an empty string or dictionary. None equals `False`; a numeric major has already been excluded. Therefore, at every point reached after a successful major check, the matcher has exactly the same truth value as

```
m and m in ([s, u], not s and [t] * 3)
```

even though this identity is false on unrestricted Python tuples of values. No sentinel and no extra source guard are needed.

The proof depends on the initial context being `[[]]`, not `[0]`, and on premise contexts being freshly constructed. Silently changing either detail would invalidate this argument.

H.6 Why the relocated induction mode must be the initializer field

The corrected selector compares arity parity with `u`. In an introduction branch this field is necessarily false-valued. No false-valued JSON value compares equal to one. Thus a meaningful compound matrix, of odd arity one or three, cannot receive a progress assumption. A literal equation has even arity two and receives progress precisely for a zero-like mode such as `0`; an empty array mode chooses the zero instance. Surplus arities retain their inherited constant interpretations. Scalar matrices cannot survive the shape check discussed above.

Comparing instead with the unused `s` field would be wrong. That field is unrestricted in introduction, so choosing `s=1` would enable progress for odd-arity compound matrices. The distinction is semantic, not merely a preferred serialization.

In the computation branch `u` is truthy. Its new comparison can choose a fresh-name opening for some noncanonical scalar initializer aliases. This is handled by the inherited zero-instance discharge lemma, not by claiming that every generated open hypothesis is uniformly true. If the generated hypothesis is $H(\vec{b})$, its zero completion $H(\vec{0})$ is a computation theorem, while the old-floor validation makes the finite family of newly admitted names $\vec{b}$ fresh for the retained conclusion and context. Substituting zero for those names and cutting in the computation theorem discharges the extra hypothesis. Negative fractional names in the newly admitted interval are included separately in this finite substitution.

H.7 Conservativity and canonical completeness

The initial empty-array formula is already a theorem. For example the permitted nonempty-seed computation gives

$$C = 0, \qquad C := I(S(S0), S0; v.0).$$

Use this equation as both the equality major and the premise of transport through $F(w) \equiv (w = 0)$ to obtain $0 = 0$. Adding that fixed theorem to a proof context is conservative by substitution of its finite derivation for hypothesis uses. No new object-level truth predicate or logical axiom has been introduced.

For canonical certificates, substantive major commands retain their old fields. A computation uses the absent-major field `[]` and its existing nonempty initializer. An introduction uses absent major `[]`, with `u=0` for equation progress and `u=[]` for cases; its witness remains in `t`. The added empty major closes without a command or name allocation, so the remaining canonical proof simulation is unchanged. First applying the existing certified-zero repair supplies every unrestricted computation instance. This gives a finite certificate for every canonical derivation of the inherited native calculus.

Conversely, the invariant makes each reached matcher a legitimate guarded matcher. The remaining commands use the inherited transport, closure, paired-introduction/elimination, and zero-instance computation arguments, with the corrected equation-only mode test. Discharging the initial $0 = 0$ theorem yields a derivation in that same calculus. Its established whole-domain iPRA interpretations therefore transfer without changing the numerical domain, equality, or object language. The new result concerns a proof-checker presentation, not a new primitive-recursive arithmetic foundation.

H.8 Raw JSON types and the generic profile

The strict profiles recognize only Python lists as containers. The short predicate `a * 0 == []` has this effect for JSON values: lists pass; strings and numbers give different zero products; dictionaries and null raise. Its numerical fallback rejects strings. The root invariant covers all numbers and Booleans that reach it, including nonfinite values accepted by the default Python loader.

The generic profile has the inherited additional aliases. An empty string and an empty dictionary open to `[]`; a dictionary with only the empty-string key opens to `[[]]`, with its value ignored. A truthy dictionary cannot survive export as a major because `m * 3` raises. Nonempty strings and nonempty dictionary keys have no finite successful opening. Actual CPython rejects them through its recursion limit and subsequent type failure, while an ideal unlimited-stack execution can diverge. The raw soundness argument above covers every finite successful run of this profile; it does not assert ideal termination on every malformed finite JSON input.

Larger arrays remain arrays throughout opening, irrespective of their ignored annotations or depth increments. They therefore do not threaten the scalar-versus-list invariant. Their formula meanings and freshness behavior remain subject to the inherited sorted-decoder proof. Physical time, memory, and stack limits can reject sufficiently large genuine proofs in every profile; the completeness claim has the same adequate-resource, canonical-proof interpretation as its predecessors.

References

[1] VladimirReshetnikov/ProveIt, *Current draft corpus*, commit `b9c4d74a0018bdafc5bc06e755a0145f4ed2b8d6`, 10 November 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/b9c4d74a0018bdafc5bc06e755a0145f4ed2b8d6/docs/drafts.

[2] ProveIt draft corpus, *Complementary introductions and generated assumptions*, inherited native-basis source preserved in the monus archive as `reference/initialized_original/inherited_choice_basis.tex`.

[3] ProveIt draft corpus, *Scalar predecessor bootstrap*, source preserved in the monus archive as `reference/initialized_original/scalar_predecessor.tex`.

[4] ProveIt draft corpus, *Equation induction and equality reflection*, source preserved in the monus archive as `reference/initialized_original/equality_reflection.tex`.

[5] ProveIt draft corpus, *Primitive-recursion compilation and the ordinary bridge*, sources preserved in the monus archive under `reference/initialized_original/`.

[6] ProveIt, *Constructing computation sources for iPRA*, `ipra_constructed_computation_2026-11-10.zip`, in the pinned corpus.

[7] ProveIt, *A one-state recursor for iPRA*, `ipra_transient_seed_2026-11-10.zip`, in the pinned corpus.

[8] ProveIt, *Equality from monus and neutral major premises*, `ipra_monus_and_neutral_major_2026-11-10.zip`, in the pinned corpus.

[9] ProveIt, *A certified zero removes a computation opcode*, `ipra_certified_zero_2026-11-10.zip`, and *Uniform zero and constructed premises*, `ipra_uniform_zero_2026-11-10.zip`, in the pinned corpus.